

ID3802 – Monthly Progress Report (March)

1 Work done in the current month

1.1 Lemma 5 of Peters

Lemma 5 of [1] acts as a vital step to show a constant bound for the competitive ratio for the algorithm proposed by Jannik Peters. This ensures that the competitive-ratio will have a constant bound of 3 and leaves room for further improvement.

Lemma 1.1. *Given a set of agents A , objects O and a partial preference profile $\succ$ in the next-best query model and an NRM M for the problem, $|rev(a)| \geq r_a - 1$ for all agents a matched to a revealed preference profile by M . Here r_a is the iteration at which agent a is no longer even.*

We assume to the contrary that there exists an agent a with $M(a) \in rev(a)$ and $|rev(a)| < r_a - 1$. We know that $a \in \mathcal{E}_{r_a-1}$ by the definition of r_a , which means that there is an even length alternating path from a to a free vertex a_k . Since a_k is free, the pair matched with a_k in M must have a rank higher than $r_a - 1$, i.e., $rank(a_k, M(a_k)) > r_a - 1$.

Assume that there exists an a_i with $i > 0$ such that $rank(a_i, M(a_{i-1})) > rank(a_{i-1}, M(a_{i-1}))$ for a_i in the even alternating path between a and a_k . Since a is even, all a_i in the path are also even in the $(r_a - 1)$ st iteration. This means that the pair matched to a_{i-1} becomes odd at the iteration at which it is matched, i.e., $M(a_{i-1}) \in \mathcal{O}_{rank(a_{i-1}, M(a_{i-1}))}$. Since the algorithm treats odd objects as inactive, they do not participate in any further path augmentations. Therefore, $M(a_{i-1})$ cannot be matched with an agent that is not a_i in G_{r_a-1} . That is, the edge $(a_i, M(a_{i-1})) \notin G_{r_a-1}$. This contradicts our assumption, and therefore, $rank(a_i, M(a_{i-1})) \leq rank(a_{i-1}, M(a_{i-1}))$ for all a_i in the alternating path with $i > 1$. Furthermore, $\succ$ can be extended so that $rank(a_1, M(a_k)) \leq r_a - 1$.

Taking into account the even length alternating path starting from a and ending at a_k , we can augment M to get a new matching M' which has $rank(a_i, M'(a_i)) = rank(a_i, M(a_{i-1})) \leq rank(a_{i-1}, M(a_{i-1}))$ for all a_i , $i > 1$ and $rank(a_1, M(a_k)) \leq r_a - 1$ while $rank(a_k, M(a_k)) > r_a - 1$ previously. Thus, M' is strictly a better rank-maximum matching compared to M . This contradicts our assumption that M is NRM and therefore $|rev(a)| \geq r_a - 1$ for all $a \in A$.

1.2 Similar arguments Observations for the RASC based algorithm

From the preliminary analysis on the performance of the RASC based algorithm, one can easily observe that the ALG is able to give a tough competition to the OPT since it gets the "skip" power like OPT thanks to the rank information as opposed to the vanilla set compare model.

Moreover, we are able to delete the Odd and Unreachable objects (permanently matched objects) from the agent - specific query subset H_i of agents due to the same reason because the edge corresponding to the next best choice of the agent from her available set of houses will only be added in the correct iteration. Thus, the size of the query set is drastically reduced, which directly influences the total number of queries asked. This property was also absent in the normal set-compare model.

What we observed in the preliminary analysis is that the argument put forward in lemma 5 can be extended to our algorithm also since the number of queries asked by the algorithm is r_a and the lemma forces the OPT to be at least $r_a - 1$. This gives us a constant bound of 3 for α .

These observations will be helpful for the later part of the analysis to derive a tighter bound.

1.3 Implementation

We have initialized an exhaustive **repository** for Necessarily Rank-Maximal matchings which documents the required background and our research work on RASC-model and the original algorithm based on it. We have also implemented the online algorithm in python in a sandbox environment using Networkx package.

Implementation details :

- The library (/implementation/lib) has two modules :
 - `graph-utils.py` for basic functions on bipartite graphs like construction, updation, computing maximum matching etc.

- `edmonds-gallai.py` contains the function for computing Edmonds - Gallai decomposition by taking a graph and a maximum matching on it as input
- `/implementation/algo.py` uses the modules to implement the RASC-based algorithm for eliciting rank-maximal matchings. It queries user preferences in the run-time.

The correctness of the code has been tested using a variety of preference profiles as inputs. Additionally, the algorithm is now able to catch an agent if she gives **inconsistent/fake preferences** and raises exception. This is the first step to set up the backend engine and to develop the frontend interface of the simulation.

The tentative tech stack for implementing the simulator is as follows :

- FastAPI (Python) with Networkx for backend
- ReactJS for the frontend
- The MVP (Minimum Viable Product) plans to use in-memory dictionaries for both state management and data storage

The github repository link : **Necessarily rank-maximal matchings repo**

2 Feedback from the mentor *Satisfactory*

3 Signature of the mentor *Pratik Ghosal*

References

31/03/2026

- [1] Jannik Peters. Online elicitation of necessarily optimal matchings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5164–5172, 2022.